

Assignment

Temporary Tables & Views

Hands-on & Scenario-Based

Q1. Create a Temporary Table from Query

Create a TEMP table named temp_top_students that contains student_id, full_name, and cgpa for all students with cgpa >= 7.5. Then SELECT from it ordered by cgpa DESC.

```
206 CREATE TEMP TABLE temp_top_students AS
207 SELECT
208     student_id,
209     student_name,
210     cgpa
211 FROM academics.students
212 WHERE cgpa >= 7.5;
213
214 SELECT *
215 FROM temp_top_students
216 ORDER BY cgpa DESC;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	cgpa numeric (4,2)
1	3	Priya Verma	9.12
2	1	Sunakshi	8.90
3	2	Reetiks	8.65
4	2001	Rahul Sharma	8.50
5	4	Vanshika	7.89

Q2. TEMP Table + JOIN

Using the temp_top_students table from Q1 (recreate if needed), join it with enrollments and courses to list each top student's courses and grades.

```
220 SELECT
221     t.student_id,
222     t.student_name,
223     t.cgpa,
224     c.course_name,
225     e.grade
226 FROM temp_top_students t
227 JOIN academics.enrollments e
228     ON t.student_id = e.student_id
229 JOIN academics.courses c
230     ON e.course_id = c.course_id
231 ORDER BY t.cgpa DESC, t.student_name;
232
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	cgpa numeric (4,2)	course_name character varying (100)	grade character (2)
1	3	Priya Verma	9.12	Database	B
2	3	Priya Verma	9.12	Operating Systems	A
3	3	Priya Verma	9.12	Database Management Syste...	B+
4	1	Sunakshi	8.90	Database Management Syste...	A
5	1	Sunakshi	8.90	Computer Networks	B+
6	1	Sunakshi	8.90	Operating Systems	A
7	2	Reetiks	8.65	Database	A
8	2	Reetiks	8.65	Computer Networks	B
9	2	Reetiks	8.65	Database Management Syste...	A+
10	4	Vanshika	7.89	Database Management Syste...	A
11	4	Vanshika	7.89	Computer Networks	B+
12	4	Vanshika	7.89	Operating Systems	A+

Q3. Session Scope of TEMP Tables

- (a) Create a TEMP table temp_demo with a single column id and insert one row.
- (b) In a comment, explain what happens if you open a brand-new Query Tool tab (new session) and try to SELECT from temp_demo.
- (c) DROP the temp table in the original session.

(a) Create TEMP table and insert one row

```
235 CREATE TEMP TABLE temp_demo (  
236     id INT  
237 );  
238  
239 INSERT INTO temp_demo (id)  
240 VALUES (1);  
241  
242 SELECT *  
243 FROM temp_demo;
```

Data Output Messages Notifications

	id
1	1

(b) New Query Tool tab/session

```
235 CREATE TEMP TABLE temp_demo (  
236     id INT  
237 );  
238  
239 INSERT INTO temp_demo (id)  
240 VALUES (1);  
241  
242 SELECT *  
243 FROM temp_demo;
```

Data Output Messages Notifications

ERROR: relation "temp_demo" already exists
SQL state: 42P07

(c) Drop it in original session

```
245  
246 DROP TABLE temp_demo;  
247
```

Data Output Messages Notifications

DROP TABLE
Query returned successfully in 116 msec.

Q4. Create a Simple View

Create a view academics.v_students_with_dept that returns student_id, full_name, cgpa, and dept_name (LEFT JOIN so students without department still appear).

```
250 CREATE VIEW academics.v_students_with_dept AS
251 SELECT
252     s.student_id,
253     s.student_name,
254     s.cgpa,
255     d.dept_name
256 FROM academics.students s
257 LEFT JOIN academics.departments d
258     ON s.dept_id = d.dept_id;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 110 msec.

Check it

```
261 SELECT *
262 FROM academics.v_students_with_dept;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	cgpa numeric (4,2)	dept_name character varying (100)
1	9999	Test Student	[null]	Computer Science
2	1	Sunakshi	8.90	Computer Science
3	4	Vanshika	7.89	Computer Science
4	2	Reetiks	8.65	Computer Science
5	3	Priya Verma	9.12	Computer Science
6	2001	Rahul Sharma	8.50	[null]

Q5. Query and Filter a View

Using the view from Q4, list students whose cgpa is above 8.0, ordered by cgpa descending.

```
266 SELECT *
267 FROM academics.v_students_with_dept
268 WHERE cgpa > 8.0
269 ORDER BY cgpa DESC;
```

Data Output Messages Notifications				
	student_id integer	student_name character varying (100)	cgpa numeric (4,2)	dept_name character varying (100)
1	3	Priya Verma	9.12	Computer Science
2	1	Sunakshi	8.90	Computer Science
3	2	Reetiks	8.65	Computer Science
4	2001	Rahul Sharma	8.50	[null]

Q6. Aggregated View

Create a view academics.v_dept_stats that shows for every department: dept_name, student_count, and avg_cgpa (rounded to 2 decimals). Include departments with zero students.

```
272 CREATE VIEW academics.v_dept_stats AS
273 SELECT
274     d.dept_name,
275     COUNT(s.student_id) AS student_count,
276     ROUND(COALESCE(AVG(s.cgpa), 0), 2) AS avg_cgpa
277 FROM academics.departments d
278 LEFT JOIN academics.students s
279     ON d.dept_id = s.dept_id
280 GROUP BY d.dept_id, d.dept_name;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 150 msec.

Check It

```
283 SELECT *
284 FROM academics.v_dept_stats
285 ORDER BY dept_name;
```

Data Output Messages Notifications

SQL

	dept_name character varying (100)	student_count bigint	avg_cgpa numeric
1	Civil Engineering	0	0.00
2	Computer Science	5	8.64
3	Electronics Engineering	0	0.00
4	Mechanical Engineering	0	0.00

Q7. Complex Multi-join View

Create a view academics.v_enrollment_report that returns:

student_name, student_dept, course_name, course_dept, grade, enroll_date

Join students → enrollments → courses → departments (twice — once for student dept, once for course dept).

```
288 CREATE VIEW academics.v_enrollment_report AS
289 SELECT
290     s.student_name AS student_name,
291     sd.dept_name AS student_dept,
292     c.course_name,
293     cd.dept_name AS course_dept,
294     e.grade,
295     e.enroll_date
296 FROM academics.students s
297 JOIN academics.enrollments e
298     ON s.student_id = e.student_id
299 JOIN academics.courses c
300     ON e.course_id = c.course_id
301 LEFT JOIN academics.departments sd
302     ON s.dept_id = sd.dept_id
303 LEFT JOIN academics.departments cd
304     ON c.dept_id = cd.dept_id;
305
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 307 msec.

Check It:

```
308 SELECT *
309 FROM academics.v_enrollment_report;
```

	student_name character varying (100)	student_dept character varying (100)	course_name character varying (100)	course_dept character varying (100)	grade character (2)	enroll_date date
1	Sunakshi	Computer Science	Database Management Syste...	Computer Science	A	2026-08-01
2	Sunakshi	Computer Science	Computer Networks	Mechanical Engineering	B+	2026-08-02
3	Sunakshi	Computer Science	Operating Systems	Mechanical Engineering	A	2026-08-03
4	Reetiks	Computer Science	Database Management Syste...	Computer Science	A+	2026-08-01
5	Reetiks	Computer Science	Computer Networks	Mechanical Engineering	B	2026-08-02
6	Reetiks	Computer Science	Database	Mechanical Engineering	A	2026-08-03
7	Priya Verma	Computer Science	Database Management Syste...	Computer Science	B+	2026-08-04
8	Priya Verma	Computer Science	Operating Systems	Mechanical Engineering	A	2026-08-05
9	Priya Verma	Computer Science	Database	Mechanical Engineering	B	2026-08-06
10	Vanshika	Computer Science	Database Management Syste...	Computer Science	A	2026-08-04
11	Vanshika	Computer Science	Computer Networks	Mechanical Engineering	B+	2026-08-05
12	Vanshika	Computer Science	Operating Systems	Mechanical Engineering	A+	2026-08-06

Q8. CREATE OR REPLACE VIEW

Modify the view academics.v_students_with_dept (from Q4) to also include the email column. Use CREATE OR REPLACE VIEW. Then query the view to confirm the new column appears.

```
312 CREATE OR REPLACE VIEW academics.v_students_with_dept AS
313 SELECT
314     s.student_id,
315     s.student_name,
316     s.cgpa,
317     d.dept_name,
318     s.email
319 FROM academics.students s
320 LEFT JOIN academics.departments d
321     ON s.dept_id = d.dept_id;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 106 msec.

Check it

```
324 SELECT *
325 FROM academics.v_students_with_dept;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	cgpa numeric (4,2)	dept_name character varying (100)	email text
1	9999	Test Student	[null]	Computer Science	test9999@example.c...
2	1	Sunakshi	8.90	Computer Science	sunakshi@gmail.com
3	4	Vanshika	7.89	Computer Science	vanshika@gmail.com
4	2	Reetiks	8.65	Computer Science	reetika@gmail.com
5	3	Priya Verma	9.12	Computer Science	priya.verma@gmail.c...
6	2001	Rahul Sharma	8.50	[null]	rahul2001@gmail.com

Q9. Updatable View Test

- (a) Create a simple view academics.v_student_cgpa that selects only student_id, full_name, cgpa from students (no joins, no aggregates).
- (b) UPDATE the view to change the cgpa of one student.
- (c) Verify the change appears in the base student's table.
- (d) In 1-2 sentences explain why this view is updatable while v_enrollment_report is not.

(a) Create the view

```
328 CREATE VIEW academics.v_student_cgpa AS
329 SELECT
330     student_id,
331     student_name,
332     cgpa
333 FROM academics.students;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 101 msec.

Check It:

```
336 SELECT *
337 FROM academics.v_student_cgpa;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	cgpa numeric (4,2)
1	3	Priya Verma	9.12
2	2	Reetiks	8.65
3	4	Vanshika	7.89
4	1	Sunakshi	8.90
5	9999	Test Student	[null]
6	2001	Rahul Sharma	8.50

(b) UPDATE the view

```
340 UPDATE academics.v_student_cgpa
341 SET cgpa = 8.50
342 WHERE student_id = 1;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 145 msec.

(c) Verify base table

345	SELECT
346	student_id,
347	student_name,
348	cgpa
349	FROM academics.students
350	WHERE student_id = 1;

Data Output	Messages	Notifications
		
		

	student_id [PK] integer	student_name character varying (100)	cgpa numeric (4,2)
1	1	Sunakshi	8.50

(d) Explanation

- v_student_cgpa is updatable because it is a simple view based on one base table without joins, aggregates, GROUP BY, DISTINCT, etc.
- v_enrollment_report is not directly updatable because it combines multiple tables using joins, so PostgreSQL cannot treat a row as belonging directly to one base table.

Q10. TEMP Table for Multi-step Analysis

Using only TEMP tables (no CTEs), perform these steps:

1. Create temp_enroll_count → student_id + number of courses enrolled.
2. Create temp_high_grades → student_id + count of A/A+ grades.
3. Join both temp tables with students to produce a final report: full_name, courses_taken, high_grade_count. Include students with zero enrollments (show 0s).

1. Course count

```
353 CREATE TEMP TABLE temp_enroll_count AS
354 SELECT
355     s.student_id,
356     COUNT(e.course_id) AS courses_taken
357 FROM academics.students s
358 LEFT JOIN academics.enrollments e
359     ON s.student_id = e.student_id
360 GROUP BY s.student_id;
```

Data Output Messages Notifications

SELECT 6

Query returned successfully in 140 msec.

Check It:

```
363 SELECT *
364 FROM temp_enroll_count
365 ORDER BY student_id;
```

Data Output Messages Notifications

	student_id integer	courses_taken bigint
1	1	3
2	2	3
3	3	3
4	4	3
5	2001	0
6	9999	0

2. A/A+ count

```
368 CREATE TEMP TABLE temp_high_grades AS
369 SELECT
370     s.student_id,
371     COUNT(
372         CASE
373             WHEN e.grade IN ('A', 'A+') THEN 1
374         END
375     ) AS high_grade_count
376 FROM academics.students s
377 LEFT JOIN academics.enrollments e
378     ON s.student_id = e.student_id
379 GROUP BY s.student_id;
```

Data Output Messages Notifications

SELECT 6

Query returned successfully in 142 msec.

Check:

```
382 SELECT *
383 FROM temp_high_grades
384 ORDER BY student_id;
```

Data Output Messages Notifications

	student_id integer	high_grade_count bigint
1	1	2
2	2	2
3	3	1
4	4	2
5	2001	0
6	9999	0

3. Final report

```
388 SELECT
389     s.student_name,
390     te.courses_taken,
391     th.high_grade_count
392 FROM academics.students s
393 LEFT JOIN temp_enroll_count te
394     ON s.student_id = te.student_id
395 LEFT JOIN temp_high_grades th
396     ON s.student_id = th.student_id
397 ORDER BY s.student_name;
```

Data Output Messages Notifications

	student_name character varying (100)	courses_taken bigint	high_grade_count bigint
1	Priya Verma	3	1
2	Rahul Sharma	0	0
3	Reetiks	3	2
4	Sunakshi	3	2
5	Test Student	0	0
6	Vanshika	3	2

Q11. View vs TEMP vs CTE Decision

For each scenario below, write which tool you would choose (TEMP table / View / CTE) and one short reason:

- (a) A daily dashboard query that many users will run — needs consistent logic.

Answer: View

because the logic is defined once and can be reused by many users/queries while keeping consistent logic.

- (b) A one-time complex report with 4 intermediate steps inside a single query.

Answer: CTE

because the intermediate steps are needed only inside one query and do not need to remain afterward.

- (c) ETL process with intermediate datasets, indexes and multiple queries.

Answer: TEMP TABLE

because intermediate data needs to survive across multiple statements in the same session and can be indexed for further processing.

Q12. Cleanup and Best Practice

- (a) DROP the temporary tables you created in this assignment.

- (b) DROP the views you no longer need (or keep the useful ones — your choice).

- (c) Write a short note (2-3 sentences) on when you would prefer a permanent table + scheduled job over a TEMP table for intermediate results.

(a) Drop temporary tables

```
400 DROP TABLE IF EXISTS temp_top_students;
401 DROP TABLE IF EXISTS temp_enroll_count;
402 DROP TABLE IF EXISTS temp_high_grades;
```

Data Output Messages Notifications

```
NOTICE: table "temp_top_students" does not exist, skipping
NOTICE: table "temp_enroll_count" does not exist, skipping
NOTICE: table "temp_high_grades" does not exist, skipping
DROP TABLE
```

Query returned successfully in 127 msec.

(b) Drop views you don't need

```
405 DROP VIEW IF EXISTS academics.v_student_cgpa;  
406 DROP VIEW IF EXISTS academics.v_enrollment_report;  
407 DROP VIEW IF EXISTS academics.v_dept_stats;  
408 DROP VIEW IF EXISTS academics.v_students_with_dept;
```

Data Output	Messages	Notifications
DROP VIEW		
Query returned successfully in 102 msec.		

(c) Permanent table + scheduled job

I would prefer a permanent table with a scheduled job when the intermediate results need to be stored for a longer period, reused by multiple sessions/users, or available for reporting even after the current database session ends.